

面向对象编程十大问题

当类中未自主定义构造函数，compiler会提供默认构造函数，为什么，编译器会提供一个空类吗？

编译器不会提供一个空类。编译器只有在一定需要默认构造函数、默认析构函数、默认赋值操作符、默认取址操作符和const取址操作符这些函数的时候，才去创建对应的函数。

何时构造函数、析构函数应被定义为private？何时应在其中使用友元、static成员？

1. `private` 的构造函数和析构函数是程序员对对象创建与析构的接管，常常用在某些设计模式下的情况，如单例模式等；
2. 构造函数的私有化的类的设计保证了其他类不能从这个类派生或者创建类的实例（单例模式）；
3. 使用方法：通过友元类对象的方法来完成创建、使用static方法完成创建等；

为什么要引入成员初始化表？为什么初始化表执行次序只与类数据成员的定义次序相关？

- 初始化列表中初始化的顺序与声明的顺序一致，与自己声明的初始化顺序无关。
- 原则上的初始化顺序:就地初始化 > 构造函数的初始化列表 > 构造函数里的赋值（严格意义上不能成为初始化）
- [参考：c++中类的数据成员初始化次序](#)

```
1 class CString{
2     char *p;
3     int size;
4     public:
5         CString(int x):size(x),p(new char[size]) {}
6     };
```

为什么引入拷贝构造函数、=操作符重载？

当创建一个对象时，如果用一个同类型的对象对它进行初始化，则会调用拷贝构造函数。当有指针类的数据成员时，使用默认拷贝构造函数会使成员指针指向同一片内存区域，如果对一个对象操作之后修改了这块空间的内容，则另一个对象将会受到影响，这将是一个隐藏的错误，所以必要时我们要接管这一过程防止歧义。=操作符重载则可以使传进来的参数的类型更多样化，拷贝构造函数可以看作是=操作符重载中参数为同种对象引用的情形。

什么是延迟绑定？C++是如何实现虚函数的？

```
1 class Shape {
2     public:
3         virtual void draw() const = 0;
4         virtual void error(const string& msg);
5         int objectID() const;
6 };
```

晚绑定是指编译器或者解释器在运行前不知道对象的类型，使用晚绑定，无需检查对象的类型，只需要检查对象是否支持特性和方法即可。c++中晚绑定常常发生在使用 `virtual` 声明成员函数的情形中，在运行时刻确定，依据对象的实际类型(动态)确定执行的方法，具有灵活性高的优点和效率低的缺点。

我们该在何时使用 `virtual` ？

需要根据对象类型实现特定方法的多态时。如期望派生类中对父类的同名方法是重写时，在父类的方法定义前面添加 `virtual` 来保证子类对父类的该方法是重写。

`public` 继承和 `non-public` 继承的含义是？

`public` 继承方式

- 基类中所有 `public` 成员在派生类中为 `public` 属性；
- 基类中所有 `protected` 成员在派生类中为 `protected` 属性；
- 基类中所有 `private` 成员在派生类中不可访问。

`protected` 继承方式

- 基类中的所有 `public` 成员在派生类中为 `protected` 属性；
- 基类中的所有 `protected` 成员在派生类中为 `protected` 属性；
- 基类中的所有 `private` 成员在派生类中仍然不可访问。

`private` 继承方式

- 基类中的所有 `public` 成员在派生类中均为 `private` 属性；
- 基类中的所有 `protected` 成员在派生类中均为 `private` 属性；
- 基类中的所有 `private` 成员在派生类中均不可访问。

参考 [公有、非公用继承的区别](#)

为什么 `=` `()` `[]` `->` 不能作为全局函数重载？

- 大体上来讲，C++ 一个类本身对这几个运算符就已经有了相应的解释了。
- 如果将这四种符号进行友元全局重载，则会出现一些冲突
- [参考](#)

`=` 不能全局重载的原因

编译器总是优先在类内寻找赋值操作符的实现，找不到就提供一个默认版本，其优先级均高于全局重载的操作符，因此即使全局重载了 `=` 根据优先级也永远不会被调用。

`()` 不能全局重载的原因

`()` 能表示三重含义：

- 类型转换操作符（可以重载）
- 函数调用传参（可以重载）
- 优先级转换操作符（不能重载）

[] 不能全局重载的原因

-> 不能全局重载的原因

编译器总是优先在类内寻找赋值操作符的实现，找不到就提供一个默认版本，其优先级均高于全局重载的操作符，因此即使全局重载了根据优先级也永远不会被调用。

这两个操作符在使用的时候顺序是不能改变的，而全局重载无法限定其顺序（对象名在前还是操作符在前），会导致混乱。

何时成员函数能返回 & ?

返回引用而非传值是一种提高效率的好方法，但要注意：

- 不能返回局部变量的引用，但可以利用返回值优化机制；
- 程序上下文需要对返回结果中对应原对象中的特定数据直接进行修改时；

在何时应以何种方式重载 new、delete ?

- 系统提供的 new 和 delete 操作所涉及的空间分配和释放是通过系统的堆区管理系统来进行的，效率常常不高，对操作符 new 和 delete 进行重载，使得程序能以自己的方式来实现动态对象空间的分配和释放功能。在重载后系统不会再行调用系统默认实现的 delete ；
- [C++中重载new和delete的使用](#)